Hierarchical Semantic Graph Reasoning for Train Component Detection

Cen Chen, Kenli Li, *Senior Member, IEEE*, Xiaofeng Zou, Zhongyao Cheng, Wei Wei, *Senior Member, IEEE*, Qi Tian, *Fellow, IEEE*, and Zeng Zeng, *Senior Member, IEEE*

***Abstract*— Recently, deep learning-based approaches have achieved superior performance on object detection applications. However, object detection for industrial scenarios, where the objects may also have some structures and the structured patterns are normally presented in a hierarchical way, is not well investigated yet. In this work, we propose a novel deep learning-based method, hierarchical graphical reasoning (HGR), which utilizes the hierarchical structures of trains for train component detection. HGR contains multiple graphical reasoning branches, each of which is utilized to conduct graphical reasoning for one cluster of train components based on their sizes. In each branch, the visual appearances and structures of train components are considered jointly with our proposed novel densely connected dual-gated recurrent units (Dense-DGRUs). To the best of our knowledge, HGR is the first kind of framework that explores hierarchical structures among objects for object detection. We have collected a data set of 1130 images captured from moving trains, in which 17 334 train components are manually annotated with bounding boxes. Based on this data set, we carry out extensive experiments that have demonstrated our proposed HGR outperforms the existing state-of-the-art baselines significantly. The data set and the source code can be downloaded online at https://github.com/ChengZY/HGR.**

***Index Terms*— Automated industrial application, deep learning, hierarchical graphical reasoning (HGR), object detection.**

I. Introduction

In RECENT years, deep learning-based approaches have achieved significant performance for object detection tasks [1]. Most existing deep learning-based methodologies for object detection only extract hidden features of independent regions by convolutional neural networks (CNNs) and then conduct classification and regression for these regions independently [1]–[5]. Nevertheless, unlike the popular data sets for generic object detection, e.g., ImageNet [6] and PASCAL VOC [7], objects existing in industrial scenarios usually present some strong structured layouts [8]. For instance, a wheel shall contain a bearing during the normal operation of the train. Due to the scarceness of well-annotated data sets from the industrial area, the study of leveraging on the structural knowledge for object detection is not investigated well yet in the literature. Moreover, some previous work has shown that commonsense knowledge (e.g., scene context and object relationships) is essential to human for object recognition as well [9]–[11], and ignoring the commonsense knowledge may reduce the accuracy of object detection [12]–[14].

In this work, we make an attempt to study a challenging problem of train component detection, which is vital for several applications, including train fault detection, visual surveillance, and predictive maintenance [15], [16]. We collect and release a data set that contains 1130 images with the resolution of 1400×1024 captured from trains and a total of 17 334 manually annotated components, which belongs to 23 different categories.

Through a detailed analysis of the data set, we have found that some challenges and characteristics exist in train component detection. First, the scale variations of different train components are large, and especially, detecting small objects is one of the most important factors that affect the performance of object detection frameworks [17], [18]. As shown in Fig. 1, the nut is much smaller than other components and, thus, usually leads to components undetected. In addition, due to the fact that cameras and railway settings are usually fixed, the same train components usually present similar sizes across multiple images.

Moreover, there may exist structured patterns between different components in different scene contexts. For instance, as shown in Fig. 1(a), given the scene context of the entire image, we can infer that the bearing is more likely to be there. There are also strong co-occurrences among certain components. For instance, if there exists a wheel in the image, it is more likely that a plate near the wheel can be detected. Most importantly, such a structured pattern is usually presented in a hierarchical way. For example, in Fig. 1(b), it is likely that

Manuscript received 9 December 2019; revised 11 July 2020 and 12 December 2020; accepted 26 January 2021. Date of publication 31 March 2021; date of current version 2 September 2022. This work was supported in part by the National Natural Science Foundation of China under Grant 61902120, in part by the National Key Research and Development Program of China under Grant 2018YFB1003401, and in part by the Postdoctoral Science Foundation of China under Grant 2019M662768 and Grant 2019TQ0086. *(Corresponding authors: Kenli Li; Zeng Zeng.)*

Cen Chen, Zhongyao Cheng, and Zeng Zeng are with the Institute for Infocomm Research, Singapore 138632 (e-mail: chenc@i2r.a-star.edu.sg; cheng_zhongyao@i2r.a-star.edu.sg; zengz@i2r.a-star.edu.sg).

Kenli Li and Xiaofeng Zou are with the College of Information Science and Engineering, Hunan University, Changsha 410082, China (e-mail: lkl@hnu.edu.cn).

Wei Wei is with the School of Computer Science and Engineering, Xi'an University of Technology, Xi'an 710048, China (e-mail: weiwei@xaut.edu.cn).

Qi Tian is with Noah's Ark Lab, Huawei, Hong Kong, SAR, China (e-mail: tian.qi1@huawei.com).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TNNLS.2021.3057792.

Digital Object Identifier 10.1109/TNNLS.2021.3057792

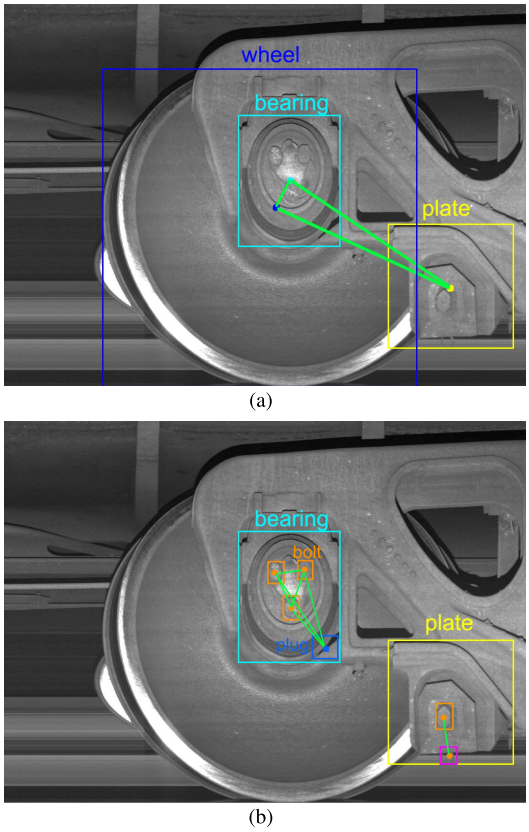


Fig. 1. Some illustrative examples of train images. (a) Illustration of the structure of large train components. For instance, if there exists a wheel in the image, it is more likely to detect a bearing in the wheel and a plate near the wheel. (b) Illustration of the structure of small train components. For example, a bearing is likely to be detected first. Based on the results, the detection of three bolts and a plug could be much simpler.

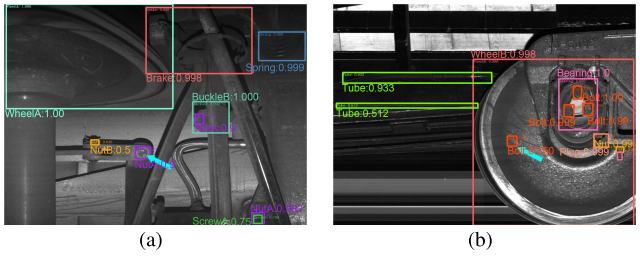


Fig. 2. Illustration of error cases made by feature pyramid network (FPN). (a) Screw is undetected. (b) Nut is wrongly detected, which does not exist.

a bearing could be detected first based on which the detection of three bolts and a plug should be much easier. Moreover, in this bearing, there are also some object relationships among these three bolts and one plug. It is worth noting that objects in other industrial scenarios (e.g., machines and product lines with a large number of components) usually have large-scale variations of different components and some fixed hierarchical structures.

Fig. 2 presents two illustrative cases detected by FPN [17]. From Fig. 2(a), we can observe that a nut can be detected, but a screw in this nut fails to be detected. If we can make the nut be the local scene context of the screw and then take the local scene context into account, the performance of detecting the screw can be improved. Another case is shown in Fig. 2(b), and a nut is wrongly detected, which does not exist. As visual features of this detected area are similar to visual features of a nut, FPN that only focuses on the object's visual appearance misidentifies the area like a nut. However, with the structure of a train, the nut should not exist at that particular position of the wheel. If object relationships and spatial dependences of these objects are taken into account, the above detection errors could be avoided, and hence, the performance of object detection could be improved.

Recent attempts of leveraging on the knowledge to improve the performance of CNN-based object detection framework can be mainly classified into two categories. The first category is to take advantage of scene context [12], [19], [20], while the second category is to conduct graphical reasoning utilizing object–object relationship and feature sharing mechanism [13], [14], [21]–[23]. However, to the best of our knowledge, all the graphical reasoning-based works construct fully connected (FC) graphs for all the potential objects and then conduct graphical reasoning for each pair of the objects. These methods ignore hierarchical object relationships and hierarchical scene context in real-life applications, especially in train components. Moreover, we show that a marriage of hierarchical scene context and hierarchical object relationships could improve the performance for train component detection.

In this work, we propose a novel deep learning-based approach, hierarchical graphical reasoning (HGR), which explores the hierarchical structural information, which includes hierarchical scene context and hierarchical object relationships, for train component detection. To address the challenge of large-scale variance among train components, different region proposal networks (RPNs) [1] are adopted to generate region proposals for different clusters of train components based on their sizes. Then, the generated region proposals are encapsulated into a hierarchical scene graph (HSG). Finally, hierarchical graphic reasoning is conducted. HGR consists of multiple graphical reasoning branches, in which the visual features of nodes in the graph are iteratively integrated with messages from HSGs by our proposed densely connected dual-gated recurrent unit (Dense-DGRU) in a recurrent and gated way. After that, the integrated node features by Dense-DGRU are then fed into the neural networks for bounding box regression and object classification. With these strategies, our proposed HGR outperforms the state-of-the-art baselines significantly. Such hierarchical structure knowledge also widely exists in industrial areas, e.g., machines and product lines with a large number of components. Our proposed HGR can be easily implemented in other industrial scenarios.

In summary, different from the existing methods that utilize visual appearance clues only, HGR considers the additional hierarchical structures of train components and performs HGR to enhance the efficiency of train component detection. Our contributions to this work are summarized as follows.

1) We propose a novel approach, HGR, for train component detection. Within HGR, the region proposals generated by different RPNs are encapsulated into HSGs. Based on the constructed HSGs, HGR conducts graphic reasoning hierarchically to improve the detection perfor-

mance, exploring the hierarchical structures of train components.

2) We design a novel dual GRU (DGRU) that can iteratively fuse visual appearance and messages from the constructed graphs (including scene context and component relationship) jointly in a weighted and gated style. In order to control the disturbance from the messages, a densely connected structure is introduced among different iterative steps in DGRU.
3) Extensive experiments on a new data set of train components, which contains 1130 images with a total of 17 334 manually annotated components from 23 categories, have demonstrated the effectiveness of HGR. We will release this train component data set to facilitate more future work on image recognition in industrial domains.

The remainder of the article is organized as follows. In Section II, we present an overview of work in deep learning-based object detection with relationships and scene context. Our proposed method is elaborated in Section III. Section IV presents the data set and experimental settings and evaluates the proposed method with respect to different metrics. Finally, we conclude our work and present the future work in Section V.

II. Related Work

Recently, deep learning techniques have achieved superior performance in various areas, such as computer vision [1], [24] and natural language processing [25]. Moreover, some work also connects deep learning with other mathematics and applications, such as tokamak [26] and fuzzy stochastic processes [27]. In this section, we first present a brief review of recent deep learning-based methods for generic object detection. Then, we discuss recent work which leverages on scene context and object relationship to improve the performance of deep learning-based object detection.

A. Generic Object Detection

Generic object detection is to identify objects in an image and then output the bounding box and the specific category of each instance detected [28]. Existing deep learning-based approaches for object detection can be mainly categorized into two classes: one-stage-based methods and two-stage-based methods.

We first describe one-stage-based approaches. The pioneering work is OverFeat [29]. Liu *et al.* [30] proposed SSD, which models the output space of bounding boxes over different aspect ratios. Fu *et al.* [31] proposed DSSD that utilizes the "SSD + Resnet101" architecture with deconvolutional layers to improve the detection performance. RetinaNet [32] finds that the foreground–background class imbalance is an important factor, which affects the detection performance for one-stage-based detectors and then designs a novel loss function by reshaping the cross-entropy loss to downweight the loss assigned to well-classified examples. YOLO [24], [33] utilizes regressions for spatially separated bounding boxes and class probabilities. DeepSaliency [34] proposes a novel multitask deep neural network model for salient object detection.

Compared with the two-stage-based counterparts, one-stage detectors are faster and simpler. However, they are reported to have poorer detection performance than two-stage-based approaches [1], [35]. A pioneer region-based convolutional network method (RCNN) was proposed in [36], which utilizes deep CNNs to classify object proposals generated by the selective search. However, RCNN is computationally expensive as the classifications for region proposals are conducted by separate CNNs. To reduce the computational cost, Girshic [37] proposed Fast RCNN that changes RCNN into a single-stage training process with shared CNNs and a multitask loss. Compared with Fast RCNN, Faster RCNN [1] adopts an RPN to generate a region proposal based on CNNs. To reduce the computational cost and storage, Wang *et al.* [4] proposed an efficient binarized object detector BiDet based on the information bottleneck (IB) principle.

B. Object Relationships for Object Detection

Generic object detection methods mentioned above usually ignore the vast amount of object relations in the real world since they leverage on visual features of objects only. To address this, some recent works aim to explore object relationships for object detection. Some pioneer studies [38], [39] leverage multimodality information to improve visual recognition. A novel framework of "knowledge-aware object detection" was proposed in [40] to integrate the external knowledge, such as knowledge graphs, into the existing object detection framework. Yang *et al.* [41] proposed Graph R-CNN that utilizes a novel relation proposal network (RePN) to detect objects and their relations in images. Liu *et al.* [13] proposed SIN that explores both scene context- and instance-level relationships for object detection. Xu *et al.* [42] proposed an iterative message passing-based mechanism to explicitly model object relationships. A spatial-aware graph relation network was proposed in [43] for large-scale object detection. A method was proposed to process a set of objects simultaneously through interactions utilizing attention mechanism [22]. Wang *et al.* [44] proposed a tensor-based multiattributes visual feature recognition method for industrial intelligence.

C. Scene Context for Object Detection

As stated in [9], [10], [45], visual objects usually coexist with context, and contextual information is essential in human object recognition. Recently, some works based on DCNNs have attempted to take advantage of contextual information for object detection. DeepIDNet [20] integrates image classification results and object detection results together to boost the object detection performance. In DeepIDNet, the whole image is regarded as the scene context of the objects in the image. Bell *et al.* [12] introduced inside–outside net that explores contextual information across entire images to improve the detection performance by the proposed spatial recurrent neural networks (RNNs). SIN [13] incorporates scene context in an iterative reasoning way based on GRU. However, these methods only consider the global scene context and ignore the hierarchical scene context. Moreover, semantic context-based

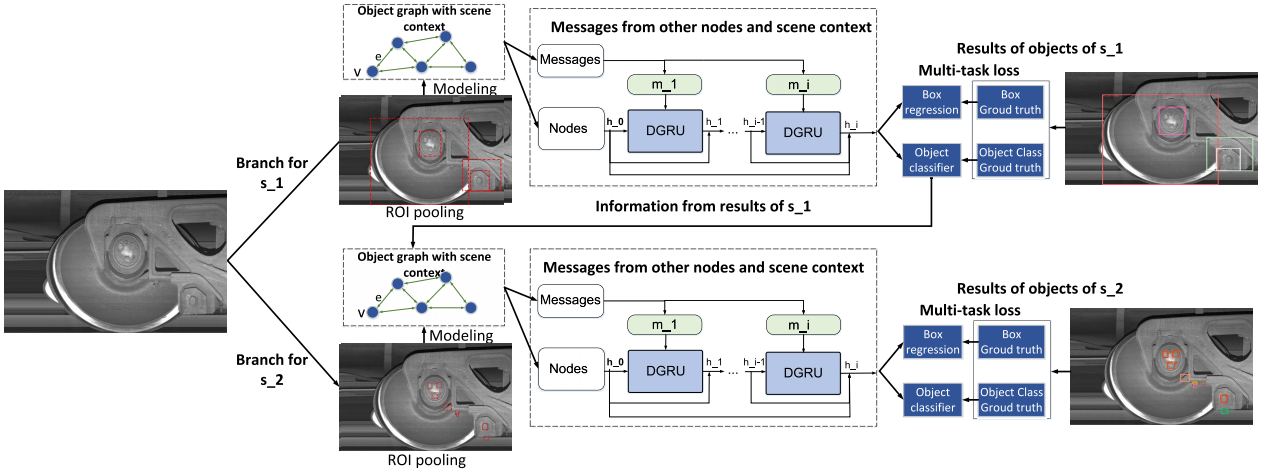


Fig. 3. Overview of our proposed HGR framework for train component detection. There are hierarchical reasoning branches in HGR. This figure takes two reasoning branches as an example. Each reasoning branch performs component detection of a specific size. The inputs of each branch are full images. In each branch, the region proposals generated by RPN are first modeled into a scene graph (SG). Then, our proposed Dense-DGRU is utilized to iteratively propagate messages, including components relationships and the scene context from the corresponding based on the constructed graph. After iterative reasoning with Dense-DGRU in each branch, the eventual integrated node features of each proposal node are used to conduct object classification and bounding box regressions utilizing the multitask loss adopted in Faster RCNN.

object detection has not been well investigated yet in industrial scenarios.

III. Methodology

The details of our proposed HGR are described in this section. First, we give a brief description of the Faster RCNN and our proposed HGR framework. Second, the process of modeling the hierarchical structures of train components with HSG is described. Third, the details of semantic graph reasoning with Dense-DGRUs are presented.

A. Overview of Faster RCNN

Faster RCNN [46] takes original images as inputs, and it outputs the detected objects with bounding boxes and object scores that denote the positions and classifications of the objects, respectively. There are two stages in Faster RCNN. The first stage is RPN, which is utilized to generate candidate object bounding boxes. The inputs of RPN are original images, and the outputs are a set of object proposals that denote the probabilities of objects [46] for each image. A region-of-interest (RoI) pool is constructed based on the object proposals. There are a feature vector and position information in each RoI proposal in the RoI pool. The second stage aims to extract features of each object proposal in the RoI pool and performs object classification and bounding-box regressions for each object proposal. In Faster RCNN, object proposals are classified separately and independently by neural networks with the visual features only.

B. Framework Overview

Our proposed HGR is built based on Faster RCNN [1] due to its effectiveness in object detection. The framework of HGR is presented in Fig. 3. As discussed above, due to the fixed distance between cameras and moving trains, train components may have relatively fixed sizes and can be classified according to the sizes first. Hence, we divide the train components into different groups based on their sizes. For the sake of simplicity, we only consider two classes of sizes in this work: large, s_1, and small, s_2. After the clustering, all the categories of train components are divided into either s_1 or s_2, which are sorted in decreasing order according to the size scores, i.e., objects in s_1 have the largest sizes. Let p_1 and p_2 be two graphical reasoning branches based on Faster R-CNN pipelines, respectively.

Each branch takes full images as inputs and performs component detection of a specific size, e.g., p_1 is for s_1 and p_2 is for s_2. In each branch, RoIs are first generated by RPN [1]. Then, RoIs in these branches are encapsulated into a hierarchical object–object relation graphs with hierarchical scene context, termed HSGs, where each branch has its own subgraph, e.g., G_i for s_i. The graphical reasoning branches are stacked hierarchically. First, within p_1, our proposed Dense-DGRU is utilized to iteratively propagate messages, including components relationships and the scene context from the subgraph of s_1. After that, eventual integrated node features are fed into neural networks for object classification and bounding box regressions for components of s_1. Our proposed Dense-DGRU is plugged in Faster RCNN [37], [46] as a demonstration. We also adopt the same multitask loss (including classification loss and bounding box regression loss) utilized in Faster RCNN to optimize the model. Subsequently, the results of p_1 are passed to p_2, and the graphical reasoning is also conducted in p_2 to predict the components of s_2.

C. Modeling

We then describe the details of modeling the hierarchical structure of train components, as shown in Fig. 4. RoIs in each branch are encapsulated into a spatial-aware object–object relation graph with scene contexts, termed SG. These RoIs interact with each other via the graph under the guidance of a

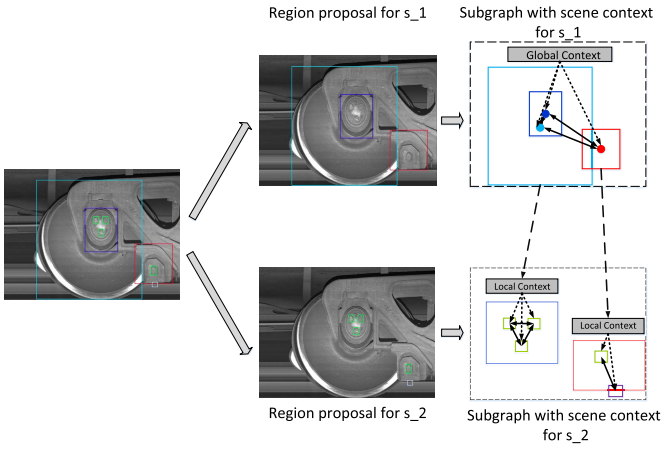


Fig. 4. Modeling the hierarchical structures into HSGs. Take two classes of sizes s_1 and s_2 as an example. The region proposals of s_1 and s_2 are modeled into two subgraphs G_1 and G_2 separately. These two subgraphs are then formed into the HSG HG, which is a collection of SGs. That is, HG $= \{G_1, \ldots, G_t\}$.

specific scene context. All the graphs form the HSGs, termed HSGs. The modeling process is described in detail as follows.

1) Spatial-Aware Object–Object Relation Graph: First, we describe the details of formulating RoIs in each branch into spatial-aware object–object relation graphs. In each branch, RPN [1] takes an original image as an input and then generates thousands of region proposals (RoIs) that may contain objects. Then, nonmaximum suppression (NMS) [47] is then adopted to choose a fixed number of RoIs. After that, we build a directed graph $G = (V, E)$ based on selected RoIs, where $v \in V$ is a node, which means an RoI, and $e \in E$ is an edge of each pair among RoIs, which denotes the relationship of each RoI pair.

As discussed above, we can classify objects into several classes. For objects that belong to class s_i, where $i = 1$ or 2 in our study, we construct a graph $G_i = (V_i, E_i)$, in which the node $v_{(i,j)} = (fv_{(i,j)}, r_{(i,j)}) \in V_i$ denotes the jth node in the RoI pool of the branch p_i, $fv_{(i,j)}$ means visual features of $v_{(i,j)}$, and $r_{(i,j)}$ is spatial information of $v_{(i,j)}$. The edge $e \in E$ stands for the pairwise relationship among RoIs. For each node $v_{(i,j)}$, we extract visual features $fv_{(i,j)}$ by an FC layer from RoI $v_{(i,j)}$. For an edge $e_{(i,j)\to(i,k)}$ from node $v_{(i,j)}$ to $v_{(i,k)}$, both the spatial information and visual features of $v_{(i,j)}$ and $v_{(i,k)}$ are utilized to compute a scalar. This scalar denotes the influence of $v_{(i,j)}$ on $v_{(i,k)}$. In this way, both visual features and spatial information of each pair of RoI can be modeled. Formally, we jointly consider relative positions and visual features to form the edge $e_{(i,j)\to(i,k)}$ as follows:

$$\begin{aligned} e_{(i,j)\to(i,k)} &= \text{relu}(W_p R_{(i,j)\to(i,k)}) * \tanh\left(W_v\left[f^v_{(i,j)}, f^v_{i,k}\right]\right) \\ R_{(i,j)\to(i,k)} &= \left[w_{(i,j)}, h_{(i,j)}, s_{(i,j)}, w_{(i,k)}, h_{(i,k)}, s_{(i,k)}, \right. \\ &\quad \left. \frac{x_{(i,k)} - x_{(i,j)}}{w_{(i,k)}}, \frac{y_{(i,k)} - y_{(i,j)}}{h_{(i,k)}}\right] \end{aligned} \tag{1}$$

where W_p is the learnable weight of the relation of relative positions, while W_v is the learnable weight of visual features, and $R_{(i,j)\to(i,k)}$ is the relative position of object (i, j) and (i, k). (x, y) is the coordinate of the center of an RoI, and w, h, and s are the width, height, and size of an RoI, respectively.

Instead of connecting all the nodes in G_i for branch p_i, a threshold can be set to make the graph sparse. This is motivated by the observation that components are always affected by the components around them, especially for the small components.

2) Integrating Scene Context With Scene Graph: We discuss the way of integrating the scene context into the spatial-aware object–object relation graphs. For ease of simplicity, the spatial-aware object–object relation graph with the scene context is termed SG. After integrating the scene context into the object–object relation graph, the formulation of each node is $v_{(i,j)} = (f_{(i,j)}, c_{(i,j)}, r_{(i,j)}) \in V_i$, where $c_{(i,j)}$ is the scene context of node $v_{(i,j)}$.

The formulations of the scene contexts for different branches are different. Using two reasoning branches as an example, RPN in p_1 branch first generates region proposals for components with s_1. Then, the SG G_1 for components with s_1 is generated. Specifically, the entire image can be regarded as the scene context of all the region proposals extracted in the p_1 branch. The results of the p_1 branch are the detected components of s_1 with the component categories and bounding box parameters. Then, the detected results of p_1 branch are sent to the p_2 branch and utilized to form the scene context for node proposals in s_2 as follows. If the region proposal $v_{(2,j)}$ in p_2 branch is in a component $o_{(1,k)}$ detected in branch p_1, $o_{(1,k)}$ is regarded as the scene context of $v_{(2,j)}$. If we cannot find a component detected in p_1 branch, the scene contexts for $v_{(2,j)}$ are set to empty. Although we find that setting two branches can lead to satisfactory results, in practice, one can stack an arbitrary number of branches in this manner to explore more complicated structured information.

3) Hierarchical Scene Graph (HSG): In HSG, HG is a collection of SGs. That is, HG $= \{G_1, \ldots, G_t\}$, where G_j is the SG of objects belongs to class s_j. Moreover, the scales of the objects in G_{j-1} are larger than that in G_j.

D. Semantic Graph Reasoning With Dense-DGRUs

We then describe the details of semantic graph reasoning with our proposed Dense-DGRUs.

1) Gated Fusion With Gated Recurrent Unit (GRU): Gated recurrent unit (GRU) [48] is a powerful variant of RNNs [26], [49], [50] for sequence learning. There are two kinds of gates in GRU: update gate and reset gate. At each iteration, new inputs and historical hidden states are fused together. During each iteration in GRU, the update gate is utilized to control the degree to which previous hidden states can be brought into current hidden states, while the reset gate is leveraged on to control the extent to which previous hidden states can be ignored. In this manner, GRUs can memorize historical information in a recurrent manner.

In our proposed methodology, we utilize GRUs to fuse visual features of RoIs with messages encoded from the hierarchical structure. Each branch of our HGR contains a GRU. In each branch, original visual features of all the nodes in a constructed SG are set as original hidden states h_0 of a

GRU unit, which are defined in (2)

$$h_0 = \left\{ f_1^v, \ldots, f_n^v \right\} \tag{2}$$

where n denotes the number of the RoIs and $f_1^v, \ldots, f_n^v$ are visual features of all the nodes.

In each iteration in a GRU, current hidden states and inputs are fused together utilizing the update gate and the reset gate in a GRU unit. In our methodology, the inputs of a GRU are set as incoming messages of a constructed SG. Through more iterations, the visual features and incoming messages are fused iteratively. Formally, the fusion process of a GRU unit at the $(t+1)$th iteration is presented in (3)

$$\begin{aligned} r_{t+1} &= \sigma(\Theta_r[m_{t+1}, h_t] + b_r) \\ u_{t+1} &= \sigma(\Theta_u[m_{t+1}, h_t] + b_u) \\ c_{t+1} &= \tanh(\Theta_c[m_{t+1}, (r_{t+1} \odot h_t)] + b_c) \\ h_{t+1} &= u_{t+1} \odot h_t + (1 - u_{t+1}) \odot c_{t+1} \end{aligned} \tag{3}$$

where h_t means hidden states of a GRU at the tth iteration and are also considered as original hidden states for the $(t+1)$th iteration, h_{t+1} denotes generated new hidden states of a GRU at the $(t+1)$th iteration, m_{t+1} denotes the incoming messages (inputs) at the $(t+1)$th iteration, r_{t+1} and u_{t+1} are the reset gate and the update gate at the $(t+1)$th iteration, respectively, $\odot$ is the operation of elementwise multiplication, and Θ_r, Θ_u, and Θ_c denote the learned parameters.

As discussed above, original hidden states h_0 are set as original visual features of all the RoIs, and m_{t+1} is set as the input information from the hierarchical structure of trains. From (3), we can observe that, at iteration step $t+1$, a GRU can integrate h_t and m_{t+1} utilizing the reset gate and update gate and then produce new hidden states h_{t+1}. As stated in [51], the update gate and the rest gate in a GRU can ignore some parts of previous hidden states that are not relative with incoming messages from an SG or use the incoming messages to enhance the hidden states. Therefore, the original visual features of all the nodes and the incoming messages from the hierarchical structure can be updated in an iterative and gated way.

2) Semantic Graph Reasoning With DGRU: In each branch, the classification of each region proposal is carried out based on visual features and messages passing from the corresponding SG. However, a GRU can only receive only one type of incoming message. To tackle this, we propose a novel DGRU scheme to receive the incoming messages of other nodes and the scene context for each SG G_i and fuse them together.

In DGRU, there are two kinds of GRUs: SGRU and RGRU in DGRU, which is shown in Fig. 5(a). SGRU aims at fusing messages from the scene context, while RGRU aims at fusing messages from object relationships. The initial hidden states of both SGRU and RGRU are original visual features of all the nodes. At each iteration, each GRU unit only receives one kind of message, fuses the received message and hidden states generated in the last iteration, and then updates the current states of nodes. Specifically, SGRU takes the encoding messages from the scene context as inputs, while RGRU takes the encoding messages from object relationships. The updated hidden states of each GRU unit are then combined together to form the updated hidden states of DGRU by our proposed gated fusion mechanism. The details of the message encoding method and gated fusion for these two kinds of hidden states are discussed in the following.

Taking SGRU as an example, the updating process of an SGRU unit $(t+1)$th iteration is formulated as follows. The updating process of an RGRU unit is similar to that of an SGRU unit

$$\begin{aligned} sr_{t+1} &= \sigma(\Theta_r[sm_{t+1}, h_t] + b_s r) \\ su_{t+1} &= \sigma(\Theta_u[sm_{t+1}, h_t] + b_s u) \\ sc_{t+1} &= \tanh(\Theta_c[sm_{t+1}, (sr_{t+1} \odot h_t)] + b_s c) \\ sh_{t+1} &= u_{t+1} \odot h_t + (1 - su_{t+1}) \odot sc_{t+1} \end{aligned} \tag{4}$$

where sm_t denotes incoming message from scene context, h_t means hidden states at the tth iteration of DGUR, sh_{t+1} is hidden states output of RGRU at the $(t+1)$th iteration, sr_{t+1} and su_{t+1} denote the reset gate and the update gate of SGRU, respectively, and Θ_{sr}, Θ_{su}, and Θ_{sc} are the learned parameters of an SGRU unit.

DGRU updates hidden states of nodes iteratively. It takes updated node states as their hidden states, also takes messages from the scene context sm, and messages from object relationships rm as inputs, and computes the following hidden states. When hidden states of objects are updated, relationships among objects and the scene context will also be updated. After some iterations, the final fused hidden states of all the nodes are then utilized for object classifications and bounding box regressions.

3) Message Encoding From Object Relationships: As discussed previously, DGRU has two different kinds of incoming messages, (i.e., rm and sm). The message encoding method for object relationships is first described. As discussed in Section III-C1, the relationships among objects are correlated with the object positions and visual features, e.g., a wheel usually contains a bearing and the bearing is the most important in the nearby components. We consider the relative visual features and object positions jointly to form the edge of each object pair. Overall, the incoming messages of RGRU can be calculated, as shown in the following:

$$\begin{aligned} m^e_{(i,j)} &= W^e_{(i,j)} \mathrm{MPool}^e_{(i,j)} \\ \mathrm{MPool}^e_{(i,j)} &= \left(e_{(i,k)\to(i,j)} f^v_{(i,k)}, (v_{(i,j)}, v_{(i,k)}) \in V_i \right) \end{aligned} \tag{5}$$

where $e_{(i,j)\to(i,k)}$ denotes the edge flowing from node $v_{(i,j)}$ to node $v_{(i,k)}$, V_i is the set of nodes in the constructed SG G_i, $W^e_{(i,j)}$ is the learnable weight for influence between node $v_{(i,j)}$ and other nodes in V_i, and $\mathrm{MPool}^e_{(i,j)}$ is the message pool for node $v_{(i,j)}$.

From the definition of messages from object relations, it is obvious that, we can utilize the visual features and spatial position relationship between objects jointly in our proposed framework. We utilize the max-pooling method to extract the most important messages from object relationships.

4) Message Encoding From the Scene Context: Each proposal node in the SG G_i also receives the messages from the scene context. The visual features of the scene context are extracted by an FC layer and considered as the incoming messages from the scene context. We also take the visual features of RoIs as initial hidden states of SGRU. An SGRU cell can utilize the update gate and the reset gate to choose to

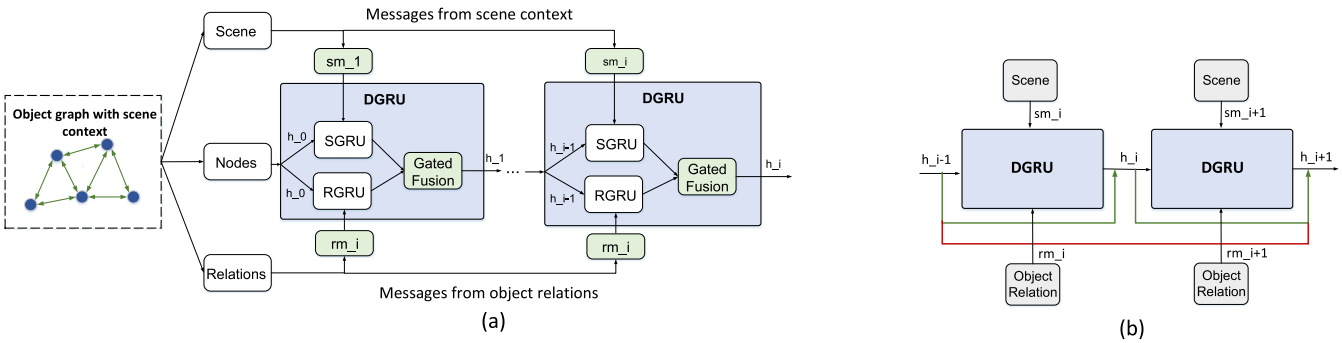


Fig. 5. (a) Structure of the proposed DGRU for semantic graph reasoning. (b) DGRU with dense connections (Dense-DGRU).

discard some parts of hidden states of RoIs that are not relative to the scene context or use the scene context to strengthen some parts of hidden states of RoIs.

5) Gated Fusion in DGRU: Each GRU unit in DGRU can only receive a kind of incoming message (i.e., from scene context or from object relationships). We introduce a gated fusion function to aggregate hidden states generated by these two kinds of incoming messages. We compute weight factors for each kind of incoming message and fuse generated hidden states using a weighted sum. The equation of the gated fusion method for integrating hidden states generated by SGRU and RGRU at time t can be formulated as follows:

$$h = W_{sh} \otimes sh + W_{rh} \otimes rh \tag{6}$$

where sh and rh are hidden states generated by SGRU and RGRU, respectively, $W_{sh}, \ldots, W_{rh}$ are learnable parameters that can adjust the degrees affected by the incoming messages from the scene context and object relationships, $\otimes$ is the elementwise multiplication operation for tensors, and h means the fused hidden states.

6) Densely Connected DGRU (Dense-DGRU): In [52], the authors introduced the dense convolutional networks (DenseNets), which connects each convolutional layer to the following convolutional layers in a dense manner, to alleviate the vanishing-gradient problem and strengthen feature propagation.

Motivated by [52], we propose a novel densely connected DGRU (Dense-DGRU). Unlike DenseNets that connect convolutional layers densely, Dense-DGRU connects the hidden states generated at each iteration densely. For each iteration in Dense-DGRU, the hidden states at all preceding iterations are added to its current hidden states. As shown in Fig. 5(b), in Dense-DGRU, a DGRU cell is encapsulated as a computational block where preceding hidden states are passed by the shortcut of the blocks. The hidden states at iteration t are formulated as follows:

$$\tilde{h}_t = \phi(\tilde{h}_{t-1}, sm_t, rm_t) + (\tilde{h}_{t-1} + \cdots + \tilde{h}_0) \tag{7}$$

where $(\tilde{h}_{t-1} + \cdots + \tilde{h}_0)$ means preceding hidden states, $\tilde{h}_t$ denotes hidden states generated at time t, and ϕ denotes the function of DGRU.

Through this densely connected mechanism, hidden states of DGRU at previous iterations are added to the following hidden states generated by DGRU directly without nonlinear activation. This densely connected mechanism for hidden states of DGRU makes visual features of the nodes (initial hidden states of DGR) have more influences on the eventual hidden states of the nodes, thus controlling the messages from the scene context and other nodes within an acceptable range.

IV. Experimental Results

A. Data Set Description

We evaluate the performance of HGR, which consists of two branches, on a new collected train component data set. The data set contains 23 categories of train components. To demonstrate the efficiency of the branches of HGR, categories of train components are divided into two classes based on their sizes: large components (i.e., holder, wheel$_a$, wheel$_b$, brake, spring, plate$_a$, plate$_b$, buc$_a$, buc$_b$, plate$_c$, plate$_d$, joint$_a$, fixator, and bearing); small components (i.e., nut$_a$, screw$_a$, nut$_b$, screw$_b$, wire, bolt, loop, joint$_b$, and plug). In the meanwhile, we randomly split the data set into training and testing data sets. There are 13 612 bounding boxes in the training data set, and there are 3722 bounding boxes in the testing data set.

B. Baselines Under Comparison

In this section, we compare our proposed HGR with other object detection frameworks as follows.

1) YOLOv3 [53] is a popular and efficient one-stage-based object detector based on DCNNs.
2) RetinaNet [54] is a one-stage detector that uses focal-loss to solve the problem of class imbalance caused by extreme foreground–background ratio.
3) FRCNN (Faster RCNN) [1] is a representative two-stage-based object detector.
4) SIN [13] leverages global scene context and object relationships for object detection. In SIN, the impacts of object relationships and global scene context are treated equally.
5) The relation network (RN) [22] exploits object relationships based on the attention mechanism.
6) FRCNNF (Faster RCNN on FPN) and FPNs [17] utilize fuse visual features in different layers and show significant improvement, especially for small object detection [17].

7) SINF, which means that we built SIN based on FRCNNF.

C. Implementation Details

We build two variants of our proposed framework: HGR and HGRF. HGR is based on FRCNN, and HGRF is based on FRCNNF. For a fair comparison, the hyperparameters of HGR mostly follow [46], while the hyperparameters of HGRF mostly follow [17]. Both of them use the Resnet 101 as their backbone. The optimizer of the models uses SGD. We use a learning rate of 0.001, weight decay is 0.1, and momentum of 0.9 for the SGD optimizer. The batch size of HGR and HGRF training and testing is 1. For RPN in both of the two architecture, the overlap threshold of the positive sample is 0.7. The NMS is applied in RPN, and its IoU threshold is 0.7. The ratio of positive and negative samples, which is used to train the RPN, is $1:3$. We set 128 as the batch size in RPN. The training epoch is 25. For HGR, the initial anchor scale in the final feature map is $\{4, 8, 16\}$, and the initial anchor ratio is $\{0.5, 1, 2\}$. For HGRF, the FPN anchor scales from the five hidden layers are set as $\{32, 64, 128, 256, 512\}$. All the models run on a single server with eight Tesla P100 cards. HGR and HGRF are both implemented with Pytorch.[1]

In order to fairly compare with all the FRCNN-based baseline, the backbone network for FRCNN, SIN, RN, FRCNNF, SINF, and our proposed HGR and HGRF is ResNet-101 [55] pretrained on ImageNet [56]. Hyperparameters of FRCNN mostly follow [46], and hyperparameters of FRCNNF mostly follow [17]. For YOLOv3, we also utilize the networks pretrained on ImageNet. In addition, in the training phase, we use NMS to select 128 boxes as region proposals for FRCNN and FPN, and in the testing phase, 300 region proposals are selected using NMS.

Both branches of p_1 and p_2 in HGR and HGRF are optimized with multitask loss (including classification loss and bounding box regression loss). To ensure fairness, the hyperparameters of the backbone network of HGR are set the same as FRCNN, and the hyperparameters of the backbone network of HGRF are set the same as FPN. During the training process of p_1, the global scene context and relationship of components with s_1 are explored to predict components of s_1. For the training process of p_2, the ground truth of p_1 is passed to p_2, and the graphical reasoning with the scene context and relationships of components with s_2 is performed to predict the components of s_2. For DGRU, the number of iterations is set as 2. The number of layers of the SGRU and RGRU in DGRU is set as 1, while the number of hidden nodes is set as 4096. These hyperparameters are set as the same as that in SIN and SINF to ensure fairness. We adopt widely utilized mean average precision (mAP) [1], [17] as the performance metric. Because training after 12 epochs for all the methods would not increase the detection accuracy, we only train 24 epochs and report the best results.

[1]Source codes can be provided on request. Source codes and the data set will be released publicly to facilitate more future work after the reviewing process.

D. Overall Performance

Like all the deep learning-based object detection models, during the training phase, all the methods in our experiments take original images in the training data set as inputs and utilize the annotated images to optimize the models. During the inference phase, all the trained models take original images in the testing data set as inputs and output the detected components with bounding boxes and object scores, which denote the positions and classifications of the objects, respectively. We report mAP scores at 0.5 for intersection-over-union (IoU) thresholds of proposals for all the methods. Table I shows the detailed detection results of all train components obtained by other baselines and our proposed HGR and HGRF. Table II summarizes the overall detection performance, where mAP denotes the results of all the train components, while mAP^s and mAP^l denote results of small and large train components, respectively. The first group of detectors in Table I is one-stage detectors, the second group is FRCNN-based detectors, and the last group is FRCNNF-based detectors. We can observe that SIN performs slightly better than FRCNN. However, frameworks based on FRCNN do not perform well on small train components detection (e.g., screw and nut).

The performance of HGR based on FRCNN is much better than other FRCNN-based baselines. The adopting of FPN significantly improves the performance, especially for small train components. Nevertheless, FRCNNF is worse than FRCNN on the detection performance of large train components. Our proposed HGRF outperforms other FRCNN-based baselines for detecting both large and small train components. Specifically, mAP^l is improved from 95.29% to 98.76%, while mAP^s is significantly improved from 72.94% to 79.59%. In addition, HGRF also outperforms all single-model detectors by a large margin, under all evaluation metrics. Overall, HGRF obtains the best detection performance.

E. Effectiveness of Graphical Reasoning

To evaluate the effectiveness of our proposed graphical reasoning mechanism, the following variants are explored.

1) HGRF-HSG means exploring hierarchical scene context only.
2) HGRF-ORG denotes exploring the hierarchical object relationships only.
3) HGRF-PORG denotes pruning the edges of SG G_2 by a threshold to make the graph sparse and then fusing the object relationships in the graph.

Table III shows the performance of our proposed HGRF and its three variants. We can observe that the three variants perform better than FRCNNF for large components. It demonstrates the effectiveness of both exploring the scene context and object relationships for train component detection. HGRF, which integrates the scene context and object relationships together, achieves the best mAP for both large and small train components. Fig. 6 illustrates some qualitative comparisons between HGRF and FRCNNF on large components.

In the following part, we will discuss the performance of HGRF and its variants on small components in detail. Compared to the baseline, the mAP of HGRF-HSG increases

TABLE I
DETECTION PERFORMANCE OF ALL THE TRAIN COMPONENTS

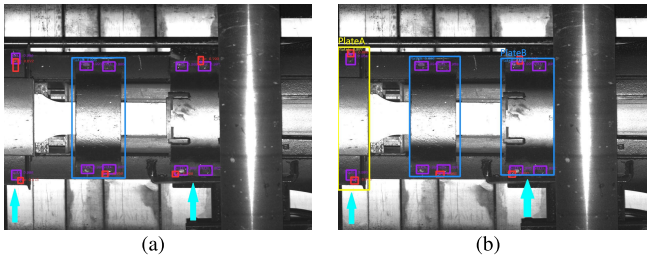
Types	Components	YOLOv3	Retina	FRCNN	SIN	RN	**HGR**	FRCNNF	SINF	**HGRF**
Large	$wheel_a$	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	**100.0**
	$wheel_b$	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	**100.0**
	brake	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	**100.0**
	spring	99.7	100.0	100.0	100.0	100.0	100.0	100.0	100.0	**100.0**
	$plate_a$	4.0	98.9	100.0	98.1	99.3	100.0	81.4	64.5	**100.0**
	$plate_b$	86.3	98.7	98.2	99.1	98.3	**99.1**	72.6	81.8	98.8
	buc_a	71.1	84.3	85.4	87.1	86.9	81.5	83.8	81.6	**88.2**
	buc_b	97.3	100.0	100.0	100.0	100.0	100.0	100.0	100.0	**100.0**
	$joint_a$	0.0	100.0	100.0	84.9	84.2	100.0	100.0	100.0	**100.0**
	fixator	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	**100.0**
	$plate_c$	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	**100.0**
	$plate_d$	99.9	100.0	100.0	100.0	100.0	100.0	100.0	100.0	**100.0**
	bearing	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	**100.0**
Small	nut_a	68.9	96.9	57.8	52.5	52.1	89.9	90.0	**90.4**	90.1
	$screw_a$	31.1	6.1	4.6	6.1	2.5	77.8	**84.9**	69.3	71.5
	nut_b	0.0	25.1	6.8	9.1	8.7	67.3	72.0	80.3	**87.6**
	$screw_b$	0.0	30.3	1.3	4.6	5.7	37.1	42.8	50.8	**69.2**
	wire	0.8	6.7	0.0	0.3	0.0	9.1	0.0	0.0	**22.8**
	bolt	40.1	100.0	0.0	9.1	10.5	100.0	100.0	100.0	**100.0**
	loop	0.0	75.1	0.0	0.0	0.0	77.5	90.4	99.5	**99.5**
	$joint_b$	0.0	76.5	74.1	76.4	66.3	66.6	**77.9**	73.4	76.2
	plug	98.5	99.1	98.9	99.4	99.6	**99.7**	98.5	99.3	99.2

TABLE II
OVERALL PERFORMANCE

Methods	*mAP*	mAP^s	mAP^l
YOLOv3	60.58	26.60	82.42
RetinaNet	82.42	57.32	98.56
FRCNN	70.50	27.05	98.43
SIN	71.27	28.63	98.68
RN	70.89	27.84	98.51
HGR	87.06	69.43	98.40
FRCNNF	86.54	72.94	95.29
SINF	86.10	73.67	94.10
HGRF	**91.26**	**79.59**	**98.76**

TABLE III
EFFECTIVENESS OF GRAPHICAL REASONING

Methods	*mAP*	mAP^l	mAP^s
FRCNNF	86.54	95.29	72.94
HGRF-HSG	89.98	97.30	76.66
HGRF-ORG	88.72	97.65	75.69
HGRF-PORG	89.86	97.65	77.84
HGRF	**91.26**	**98.76**	**79.59**



(a) (b)

Fig. 6. Qualitative result comparison on (a) FRCNNF and (b) HGRF-HSG for large train components. FRCNNF cannot detect the occluded $plate_a$ and $plate_b$. In contrast, our proposed HGRF can detect them by considering the scene context of the whole image.

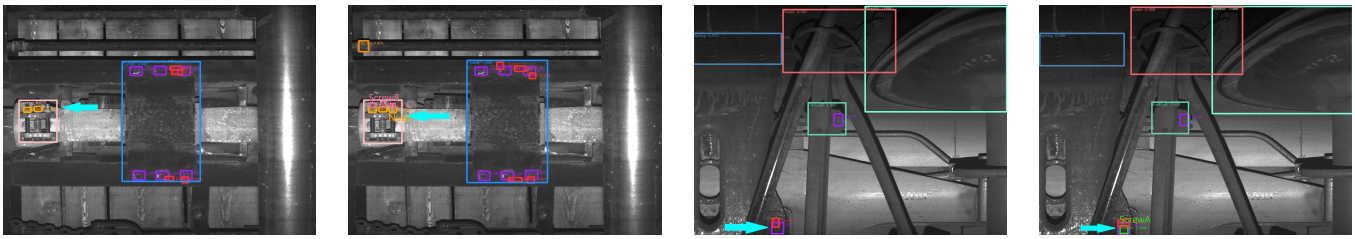
by 3.8%. This demonstrates the effectiveness of updating the object features by considering the scene contexts around the objects. HGRF-OR, which integrates the object relationship into the object's visual features, also improves the accuracy of the detection. Notice that HGRF-PORG improves the performance by more than 2% compared with HGRF-ORG. This reason is that small objects are always affected by the surrounding objects. Focusing on local object relationships around small objects is better than focusing on object relationships across all the nodes in the images. In HGRF-PORG, the edges in SG G_2 are not pruned. Therefore, mAP^l's of HGRF-PORG and HGRF-ORG are the same. Finally, we integrate HSG and PORG modules into HGRF that achieves the best detection performance for both large and small train components. Figs. 7 and 8 illustrate some qualitative comparisons of FRCNNF with HGRF-HSG and HGRF-PORG, respectively.

F. Effectiveness of Hierarchical Graphical Reasoning Methods

Four variants based on HGRF are explored in the experiments to evaluate the effectiveness of the HGR method.

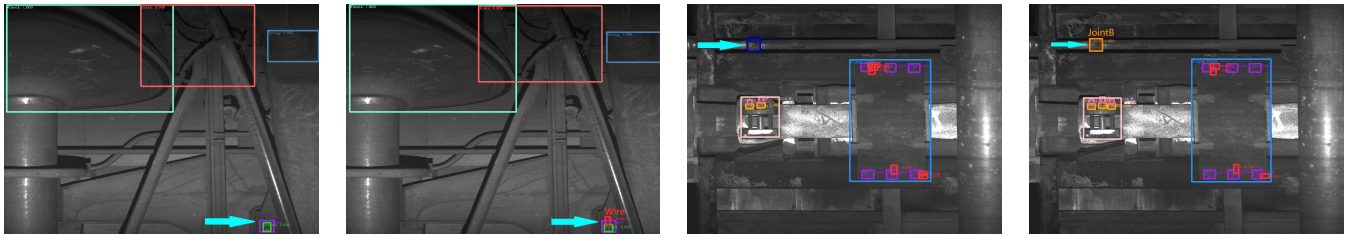
1) FRCNN-GR stands for that only an FRCNN branch with the graphical reasoning scheme is utilized.
2) HGR-NGR presents that two FRCNNF branches are adopted, but the graphical reasoning method is not adopted.
3) FRCNNF-GR denotes that only an FRCNNF branch with graphical reasoning is adopted.
4) HGRF-NGR means utilizing two FRCNNF branches without graphical reasoning.

Table IV presents the detection performance HGR, HGRF, and the above variants. We can clearly observe that graphical reasoning can improve the performance of both FRCNN and FRCNNF. Besides, the performance of detecting small train components is significantly improved by adopting the hierarchical mechanism. It is obvious that, HGR, which integrates hierarchical mechanisms with graphical reasoning, can further



(a) (b) (c) (d)

Fig. 7. Qualitative result comparison on (a) and (c) FRCNNF and (b) and (d) HGRF-HSG for small components. (1) (a) FRCNNF cannot detect nut_b and screw_b in fixator. In contrast, our proposed HGRF-HSG can detect them by exploring the scene context within fixator. (2) (c) FRCNNF cannot detect a screw_a. In contrast, our proposed HGRF-HSG integrated with the local scene of the nut_a can detect it.



(a) (b) (c) (d)

Fig. 8. Qualitative result comparison on (a) and (c) FRCNNF and (b) and (d) HGRF-ORG for small components. (1) (a) FRCNNF cannot detect a wire. In contrast, HGRF-PORG can detect it by exploring the relationship of nut, screw, and wire. (2) (b) FRCNNF mislabel joint_b as buc_a. Our proposed HGRF-PORG can detect it correctly.

TABLE IV
EFFECTIVENESS OF HGR

Methods	*mAP*	mAP^s	mAP^l
FRCNN	70.50	27.05	98.43
FRCNN-GR	72.00	30.94	98.33
HGR-NGR	84.60	64.05	97.81
HGR	87.06	69.43	98.40
FRCNNF	86.54	72.94	95.29
FRCNNF-GR	87.50	76.14	94.80
HGRF-NGR	89.06	73.76	98.70
HGRF	**91.26**	**79.59**	**98.90**

help to improve performance. In addition, some examples that illustrate the effectiveness of the hierarchical mechanism are presented in Fig. 9.

G. Evaluation of the Running Time

In this section, we study the time complexity of our proposed HGR. For simplicity, we only compare HGRF with other baselines (FPN and SINF) that have achieved relatively good results and run these models utilizing one single P100 GPU. Table V shows the average running time of one epoch training and inference of one image. HGRF-Large means the first branch in HGRF for detecting large components, while HGRF-Small means the second branch in HGRF for detecting small components. We can observe from the table that both the running times of SINF and HGRF are longer than that of FPN. The running time of one branch of HGRF with regards to training and testing is shorter than that of SINF.

H. Effectiveness of Dense-DGRU

In this experiment, we evaluate the effects of densely connected mechanisms in Dense-DGRU. One variant based on HGRF is explored in the experiments: HGRF-NDense indicates that the densely connected mechanism is not utilized. Table VI shows that HGRF-NDense can improve the detection performance compared with FRCNNF. After adopting the densely connected mechanism, the performance can be further improved. It shows the effectiveness of our proposed Dense-DGRU that can explore the structure of trains for object detection.

TABLE V
TIME COMPLEXITY OF DIFFERENT APPROACHES

Methods	Training time (s) One epoch	Testing time (s) One image
FPN	629	0.23
SINF	969	0.93
HGRF-Large	935	0.51
HGRF-Small	924	0.49

TABLE VI
EFFECTIVENESS OF DENSE-DGRU

Methods	*mAP*	mAP^s	mAP^l
FRCNNF	86.54	72.94	95.29
HGRF-NDense	88.63	75.58	97.02
HGRF	**91.26**	**79.59**	**98.76**

I. Evaluation of Aggregating Messages From Object Relationships

We evaluate the effects of different approaches of aggregating object relationships in this experiment. For each proposal node, the contribution of messages passed from other nodes is different. We need to aggregate messages from other proposal nodes. We explore two approaches of aggregating messages

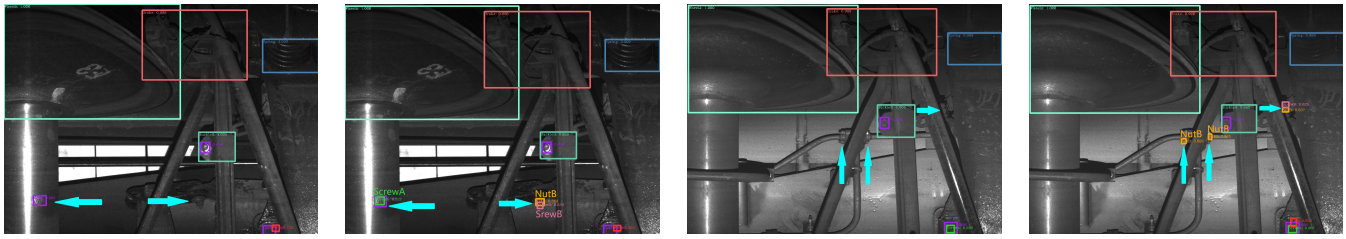


Fig. 9. Qualitative result comparison on (a) and (c) FRCNNF and (b) and (d) HGRF. HGRF uses a hierarchical mechanism to improve the recall rate of small components and further improves the accuracy of detection through graphical reasoning.

TABLE VII
EVALUATION OF AGGREGATING MESSAGES FROM OBJECT RELATIONSHIPS

Methods	*mAP*	mAP^s	mAP^l
mean-pooling	90.47	79.08	97.80
max-pooling	**91.26**	**79.59**	**98.76**

TABLE VIII
PERFORMANCE VARIATION IN DIFFERENT ITERATION STEPS

Methods	*mAP*	mAP^s	mAP^l
Step-3	88.07	73.98	97.14
Step-4	88.43	73.14	98.36
Step-2	**91.26**	**79.59**	**98.76**

from object relationships, including max-pooling and mean-pooling. From Table VII, it can be observed the max-pooling gets better performance. One possible reason is that using mean-pooling may be disturbed by a large number of objects from unrelated regions, while using max-pooling can extract the most important features.

J. Performance Under Different Iteration Steps

In this experiment, the detection performance under different numbers of iteration steps in Dense-DGRU is evaluated. In each iteration, the features of the nodes themselves are neutralized with additional messages. With more iteration steps, each node can aggregate more information from the scene context and other nodes. As shown in Table VIII, HGRF obtains the highest performance with two iteration steps. The performance is gradually decreased when increasing the number of iterations steps. That is because additional messages from the scene context and object relationships are fused well with the visual features of objects within two iteration steps. If the number of iterations steps is increased, the integration of additional incoming messages will bring noises, and more disturbance will be introduced.

V. CONCLUSION AND FUTURE WORK

This work proposes a novel deep learning-based object detector, HGR, which aims at exploring the hierarchical structure of trains for train component detection. We introduce a Dense-DGRU unit in HGR that is able to integrate visual features of train components and messages encoded from the hierarchical structure of trains in a recurrent and gated way. Evaluations on a newly collected train component data set have demonstrated that HGR outperforms existing challenging baselines significantly for train component detection.

In this work, the hierarchical structure knowledge is explored in an implicit way. We intend to construct an explicit hierarchical structural graph for the trains and explore the hierarchical structure knowledge in an explicit way in our future work.

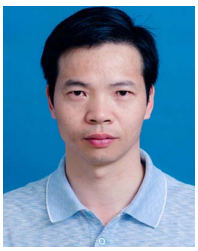
REFERENCES

[1] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Proc. NIPS*, 2015, pp. 91–99.
[2] Z. Cai and N. Vasconcelos, "Cascade R-CNN: Delving into high quality object detection," in *Proc. CVPR*, 2018, pp. 6154–6162.
[3] J. Dai, Y. Li, K. He, and J. Sun, "R-FCN: Object detection via region-based fully convolutional networks," in *Proc. NIPS*, 2016, pp. 379–387.
[4] Z. Wang, Z. Wu, J. Lu, and J. Zhou, "BiDet: An efficient binarized object detector," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2020, pp. 2049–2058.
[5] C. Chen, K. Li, S. G. Teo, X. Zou, K. Li, and Z. Zeng, "Citywide traffic flow prediction based on multiple gated spatio-temporal convolutional neural networks," *ACM Trans. Knowl. Discovery Data*, vol. 14, no. 4, pp. 1–23, Jul. 2020.
[6] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2012, pp. 1097–1105.
[7] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, "The Pascal visual object classes (VOC) challenge," *Int. J. Comput. Vis.*, vol. 88, no. 2, pp. 303–338, Jun. 2010.
[8] Z. Qin *et al.*, "Knowledge-graph based multi-target deep-learning models for train anomaly detection," in *Proc. Int. Conf. Intell. Rail Transp. (ICIRT)*, Dec. 2018, pp. 1–5.
[9] S. K. Divvala, D. Hoiem, J. H. Hays, A. A. Efros, and M. Hebert, "An empirical study of context in object detection," in *Proc. CVPR*, 2009, pp. 1271–1278.
[10] C. Galleguillos and S. Belongie, "Context based object categorization: A critical survey," *Comput. Vis. Image Understand.*, vol. 114, no. 6, pp. 712–722, Jun. 2010.
[11] M. Duan, K. Li, X. Liao, K. Li, and Q. Tian, "Features-enhanced multi-attribute estimation with convolutional tensor correlation fusion network," *ACM Trans. Multimedia Comput., Commun., Appl.*, vol. 15, no. 3, pp. 1–23, Jan. 2020.
[12] S. Bell, C. L. Zitnick, K. Bala, and R. Girshick, "Inside-outside net: Detecting objects in context with skip pooling and recurrent neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 2874–2883.
[13] Y. Liu, R. Wang, S. Shan, and X. Chen, "Structure inference net: Object detection using scene-level context and instance-level relationships," in *Proc. CVPR*, 2018, pp. 6985–6994.
[14] C. Jiang, H. Xu, X. Liang, and L. Lin, "Hybrid knowledge routed modules for large-scale object detection," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 1559–1570.
[15] X. Liu, M. R. Saat, and C. P. L. Barkan, "Analysis of causes of major train derailment and their effect on accident rates," *Transp. Res. Rec. J. Transp. Res. Board*, vol. 2289, no. 1, pp. 154–163, Jan. 2012.

[16] S. G. Chadwick, M. R. Saat, and C. P. Barkan, "Analysis of factors affecting train derailments at highway-rail grade crossings," in *Proc. Transp. Res. Board Annu. Meeting*, 2012.
[17] T.-Y. Lin, P. Dollár, R. B. Girshick, K. He, B. Hariharan, and S. J. Belongie, "Feature pyramid networks for object detection," in *Proc. ICCV*, 2017, vol. 1, no. 2, pp. 2117–2125.
[18] P. Hu and D. Ramanan, "Finding tiny faces," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 1522–1530.
[19] X. Zeng, W. Ouyang, B. Yang, J. Yan, and X. Wang, "Gated bi-directional cnn for object detection," in *Proc. ECCV*, 2016, pp. 354–369.
[20] W. Ouyang *et al.*, "Deepid-net: Deformable deep convolutional neural networks for object detection," in *Proc. CVPR*, 2015, pp. 2403–2412.
[21] X. Chen and A. Gupta, "Spatial memory for context reasoning in object detection," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 4086–4096.
[22] H. Hu, J. Gu, Z. Zhang, J. Dai, and Y. Wei, "Relation networks for object detection," in *Proc. CVPR*, 2018, vol. 2, no. 3, pp. 3588–3597.
[23] X. Chen, L.-J. Li, L. Fei-Fei, and A. Gupta, "Iterative visual reasoning beyond convolutions," in *Proc. CVPR*, 2018, pp. 7239–7248.
[24] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. CVPR*, 2016, pp. 779–788.
[25] T. Young, D. Hazarika, S. Poria, and E. Cambria, "Recent trends in deep learning based natural language processing," *IEEE Comput. Intell. Mag.*, vol. 13, no. 3, pp. 55–75, Aug. 2018.
[26] D. Rastovic, "Targeting and synchronization at tokamak with recurrent artificial neural networks," *Neural Comput. Appl.*, vol. 21, no. 5, pp. 1065–1069, Jul. 2012.
[27] D. Rastovic, "From non-Markovian processes to stochastic real time control for tokamak plasma turbulence via artificial intelligence techniques," *J. Fusion Energy*, vol. 34, no. 2, pp. 207–215, Apr. 2015.
[28] Z.-Q. Zhao, P. Zheng, S.-T. Xu, and X. Wu, "Object detection with deep learning: A review," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 11, pp. 3212–3232, Nov. 2019.
[29] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus, and Y. LeCun, "OverFeat: Integrated recognition, localization and detection using convolutional networks," in *Proc. ICLR*, 2014.
[30] W. Liu *et al.*, "SSD: Single shot multibox detector," in *Proc. ECCV*, 2016, pp. 21–37.
[31] C.-Y. Fu, W. Liu, A. Ranga, A. Tyagi, and A. C. Berg, "DSSD: Deconvolutional single shot detector," 2017, *arXiv:1701.06659*. [Online]. Available: http://arxiv.org/abs/1701.06659
[32] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 2, pp. 318–327, Feb. 2020.
[33] J. Redmon and A. Farhadi, "YOLO9000: Better, faster, stronger," in *Proc. CVPR*, 2017, pp. 7263–7271.
[34] X. Li *et al.*, "DeepSaliency: Multi-task deep neural network model for salient object detection," *IEEE Trans. Image Process.*, vol. 25, no. 8, pp. 3919–3930, Aug. 2016.
[35] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," in *Proc. IEEE Int. Conf. Comput. Vis.*, Oct. 2017, pp. 2980–2988.
[36] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *Proc. CVPR*, 2014, pp. 580–587.
[37] R. Girshick, "Fast R-CNN," in *Proc. ICCV*, 2015, pp. 1440–1448.
[38] Y. Yang, Y.-T. Zhuang, F. Wu, and Y.-H. Pan, "Harmonizing hierarchical manifolds for multimedia document semantics understanding and cross-media retrieval," *IEEE Trans. Multimedia*, vol. 10, no. 3, pp. 437–446, Apr. 2008.
[39] Y. Yang, F. Wu, F. Nie, H. T. Shen, Y. Zhuang, and A. G. Hauptmann, "Web and personal image annotation by mining label correlation with relaxed visual graph embedding," *IEEE Trans. Image Process.*, vol. 21, no. 3, pp. 1339–1351, Mar. 2012.
[40] Y. Fang, K. Kuan, J. Lin, C. Tan, and V. Chandrasekhar, "Object detection meets knowledge graphs," in *Proc. Int. Joint Conf. Artif. Intell.*, 2017.
[41] J. Yang, J. Lu, S. Lee, D. Batra, and D. Parikh, "Graph R-CNN for scene graph generation," in *Proc. ECCV*, 2018, pp. 670–685.
[42] D. Xu, Y. Zhu, C. B. Choy, and L. Fei-Fei, "Scene graph generation by iterative message passing," in *Proc. CVPR*, 2017, pp. 5410–5419.
[43] H. Xu, C. Jiang, X. Liang, and Z. Li, "Spatial-aware graph relation network for large-scale object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 9298–9307.
[44] X. Wang, L. T. Yang, L. Song, H. Wang, L. Ren, and M. J. Deen, "A tensor-based multiattributes visual feature recognition method for industrial intelligence," *IEEE Trans. Ind. Informat.*, vol. 17, no. 3, pp. 2231–2241, Mar. 2021.
[45] S. Liu, D. Huang, and Y. Wang, "Pay attention to them: Deep reinforcement learning-based cascade object detection," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 7, pp. 2544–2556, Sep. 2019.
[46] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 6, pp. 1137–1149, Jun. 2017.
[47] P. F. Felzenszwalb, R. B. Girshick, D. McAllester, and D. Ramanan, "Object detection with discriminatively trained part-based models," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 32, no. 9, pp. 1627–1645, Sep. 2010.
[48] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," 2014, *arXiv:1412.3555*. [Online]. Available: http://arxiv.org/abs/1412.3555
[49] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, Oct. 1986.
[50] C. Chen *et al.*, "Gated residual recurrent graph neural networks for traffic prediction," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, 2019, pp. 485–492.
[51] K. Cho, B. van Merrienboer, D. Bahdanau, and Y. Bengio, "On the properties of neural machine translation: Encoder-decoder approaches," 2014, *arXiv:1409.1259*. [Online]. Available: http://arxiv.org/abs/1409.1259
[52] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 4700–4708.
[53] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," 2018, *arXiv:1804.02767*. [Online]. Available: http://arxiv.org/abs/1804.02767
[54] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2980–2988.
[55] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. CVPR*, Jun. 2016, pp. 770–778.
[56] O. Russakovsky *et al.*, "ImageNet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, no. 3, pp. 211–252, Dec. 2015.

Cen Chen received the Ph.D. degree in computer science from Hunan University, Changsha, China.

He is currently working as a Scientist II with the Institute for Infocomm Research (I2R), Agency for Science, Technology and Research (A*STAR), Singapore. He has published several research articles in international conference and journals of machine learning algorithms and parallel computing, such as IEEE TRANSACTIONS ON COMPUTERS (TC), IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS (TPDS), Association for the Advancement of Artificial Intelligence (AAAI), IEEE International Conference on Data Mining (ICDM), International Conference on Parallel Processing (ICPP), and many more. His research interests include parallel and distributed computing, machine learning, and deep learning.



Kenli Li (Senior Member, IEEE) received the Ph.D. degree in computer science from the Huazhong University of Science and Technology, Wuhan, China, in 2003.

He has published more than 200 research articles in international conferences and journals, such as IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING (TKDE), IEEE TRANSACTIONS ON COMPUTERS (TC), IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS (TPDS), and International Conference on Parallel Processing (ICPP). His research interests include parallel computing, high-performance computing, and grid and cloud computing.

Dr. Li currently serves on the editorial board for the IEEE TRANSACTIONS ON COMPUTERS (TC).

Xiaofeng Zou is currently pursuing the Ph.D. degree in computer science with Hunan University, Changsha, China.

He has published several research articles in international conferences and journals, such as the Association for the Advancement of Artificial Intelligence (AAAI) and IEEE International Conference on Data Mining (ICDM). His research interests include data mining, machine learning, and deep learning.

Zhongyao Cheng is currently a Research Engineer with I2R, A*STAR, Singapore. He has provided technical support for both academic articles and industrial projects and has published several articles regarding deep learning and computer vision. His research interests include computer vision and deep learning.

Wei Wei (Senior Member, IEEE) received the Ph.D. degree from Xi'an Jiaotong University, Xi'an, China, in 2010.

He is currently an Associate Professor with the School of Computer Science and Engineering, Xi'an University of Technology, Xi'an. He has over 100 articles published or accepted by international conferences and journals. His research interests include wireless networks, sensor networks, image processing, mobile computing, distributed computing, pervasive computing, the Internet of Things, sensor data, and cloud computing. He ran many funded research projects as a principal investigator and a technical member.

Dr. Wei is a Senior Member of CCF.

Qi Tian (Fellow, IEEE) received the B.E. degree in electronic engineering from Tsinghua University, Beijing, China, in 1992, the M.S. degree in electrical and computer engineering from Drexel University, Philadelphia, PA, USA, in 1996, and the Ph.D. degree in electrical and computer engineering from the University of Illinois at Urbana-Champaign, Champaign, IL, USA, in 2002.

He is currently a Full Professor with the Department of Computer Science, The University of Texas at San Antonio (UTSA), San Antonio, TX, USA.

Zeng Zeng (Senior Member, IEEE) received the Ph.D. degree in electrical and computer engineering from the National University of Singapore, Singapore.

From 2011 to 2014, he worked as a Senior Research Fellow with the National University of Singapore. From 2005 to 2011, he worked as a Professor with the Computer and Communication School, Hunan University, Changsha, China. He is currently working as a Senior Scientist and a Program Head with I2R, A*STAR, Singapore. His research interests include distributed/parallel computing systems, data stream analysis, deep learning, multimedia storage systems, and wireless sensor networks.